

Chessy3D: A study of 3D Chessboard Detection and Pose Estimation

Luca Villani
matr. 200412

University of Modena and Reggio Emilia
269419@studenti.unimore.it

Alessandro Mezzogori
matr. 200559

University of Modena and Reggio Emilia
271617@studenti.unimore.it

Davide Della Casa Venturelli
matr. 201103

University of Modena and Reggio Emilia
71622@studenti.unimore.it

Abstract—In this paper, we present a novel framework for chessboard corners detection and chess-pieces object detection. Our approach tackles the significant challenges posed by the variability of the photos taken of the chessboard and sizes, color and differences of the chess pieces. We propose a method for chessboard keypoint detection that takes a geometric approach enhanced by the constraints in the working domain to accurately identify the keypoints on the chessboard and the squares within it. This method is robust to variations in lighting, angle, and chessboard designs. Additionally, we developed a chess-piece object detection model that can accurately detect and classify chess pieces in various conditions. Our study delves into the intricacies of chessboard and piece detection, addressing the challenges of different chessboard designs. The proposed approach uses YOLO for chess-piece detection enabling conversion from image to digital FEN annotation. Additionally we show a method for retrieving similar images using one hot encoding and a similarity search algorithm.

I. INTRODUCTION

With the rise of popularity of chess, areas such as game analysis and engine training, have gained increasing academic attention, as a result improved tooling may ease and enhance the tactical and strategic studies of the game. Chess, with its well-defined structure and semi standardized pieces, presents an ideal test bed for non trivial computer vision tasks.

This paper explores a systematic approach to the conversion from images of physical chessboards into the digital FEN representation of the captured game state, utilizing a pipeline spanning from a chessboard corners detection algorithm to chess-pieces object detection using deep learning, ending with the retrieval of similar reached positions.

Our method is designed to handle the challenges posed by the variability of chessboard photos, including different lighting conditions, angles, and chessboard designs.

We have been through a series of steps to achieve our goal:

- We started with the chessboard corners detection, in which we explored different methods both based on deep learning and fully geometric to accurately identify the keypoints on the chessboard.
- Next, we focused on chess-pieces object detection, approaching DETR and YOLO to detect and classify chess pieces in various conditions.
- Finally, we implemented an image similarity search method using one hot encoding and a similarity search algorithm.

The rest of the paper is organized as follows:

- Section II presents our proposed methods for detecting chessboard corners and cell boundaries.
- Section III describes the chess piece detection pipeline.
- Section IV outlines the procedure for generating FEN annotations from board states.
- Section V introduces the method for chess image similarity search.

II. CHESSBOARD DETECTION

Detecting a chessboard in an image involves identifying both the grid corners and the bounding boxes of all individual cells. Unlike standard calibration setups, where the chessboard is fully visible, our task required robustness to real-world conditions, including partial occlusions from chess pieces and inconsistent lighting. In these cases, lines and corners may not be clearly visible, making standard chessboard detection techniques unreliable. Over the course of this project, we evaluated a variety of techniques, refining our approach over time to improve accuracy and robustness. This section summarizes the main methods we explored, followed by a detailed description of the final solution adopted in our system.

A. Line-Based Detection

This approach aims to reconstruct the chessboard grid starting from strong edges features. After detecting line segments using the Hough Transform, we extend and merge them to improve continuity and reduce fragmentation. The goal is to recover as much of the grid structure as possible, even when parts are occluded by chess pieces.

To reduce computational complexity and increase robustness, the detected lines are clustered into vertical-ish and horizontal-ish directions using k -means based on their angles. Intersections between these classified lines are then computed to identify potential quadrilateral shapes that correspond to individual chessboard cells.

In the final stage, the algorithm searches for a regular 8x8 square pattern among the candidate quads. If some cells are missing due to occlusions, their positions are inferred by extrapolating from nearby detected cells. Once a complete grid is hypothesized, the enclosing quadrilateral is used to determine the overall chessboard corners.

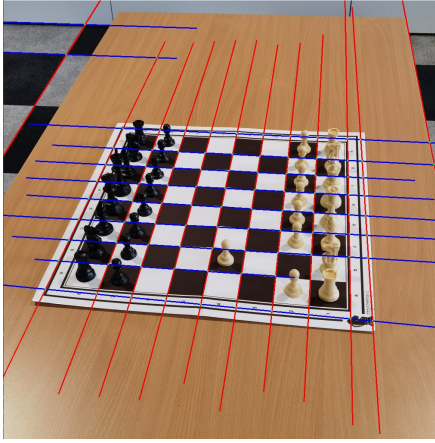


Fig. 1: Line detection and clustering



Fig. 2: Quads retrieved from the line intersections

Limitations:

- Sensitive to noise and small edge artifacts.
- Struggles with occlusions that hide or break lines.
- Requires careful tuning of thresholds and clustering parameters.
- Computationally heavy due to multiple processing stages.

B. Contour-Based Detection

This approach begins by detecting contours in the image rather than lines. The contours are approximated to polygons with fewer vertices to simplify their shapes. The polygons are then filtered based on the number of vertices (specifically looking for quadrilaterals) and their areas, aiming to identify candidate quads that correspond to chessboard cells.

Once candidate quadrilaterals are identified, we compute the smallest enclosing quadrilateral that contains all these candidate cells. This enclosing quad serves as the estimated chessboard boundary.

The cells' bounding boxes are then calculated by applying a homography transformation to the image using the chessboard corners and splitting the image into a regular 8x8 grid. This allows the algorithm to fill gaps when some cells are not

directly detected.

Limitations:

- Needs careful tuning to keep only meaningful shapes.
- Sometimes detects false quads from background clutter.
- Struggles if the chessboard is viewed at a strong angle.
- Depends on enough cells being detected to find the board boundary.

C. Parallel Multi-Parameter Contour-Based Detection

This is a variant of the Contour-Based approach where instead of using fixed parameters, a range of thresholds is tried by running multiple pipelines in parallel. Each configuration applies different Canny and Hough parameter settings, and the one that produces the highest number of valid squares is selected as the best candidate.

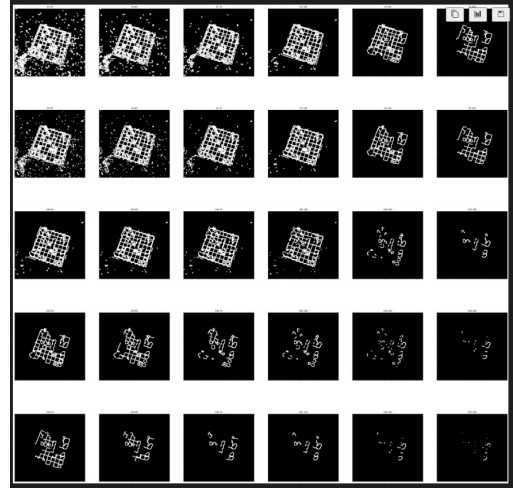


Fig. 3: Thresholding and Canny parameters exploration

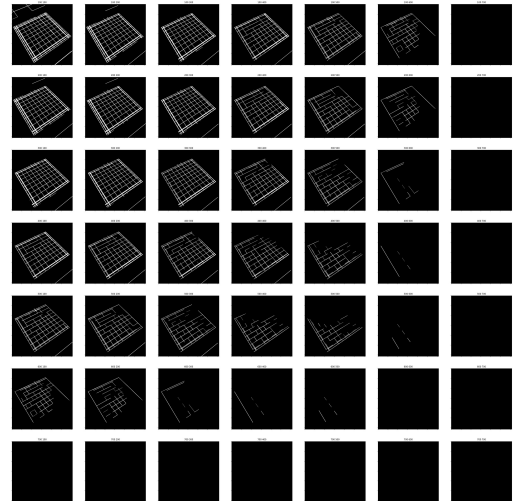


Fig. 4: Hough parameters exploration

Limitations:

- Increased computational cost due to running multiple Canny and Hough parameter sets in parallel.

- Risk of false positives if no parameter combination perfectly match the image.
- Requires a clear metric to select the optimal parameter set.
- Adds complexity in implementation and tuning compared to fixed-parameter methods.

D. Deep Learning Corner Keypoint Detection

Instead of using classical computer vision techniques, this approach uses a fine-tuned YOLOv11 pose estimation model to directly detect the four chessboard corners as keypoints. While the model achieved good quantitative metrics, the predicted keypoint locations were not always precise enough for reliable use.

The model was trained on the ChessRed2K dataset, which provides annotated corner positions for supervision. The main drawback was that the model failed to generalize to other images outside the dataset, often not detecting any corners at all. Likely, a larger and more diverse training set would be needed to make this solution viable.



Fig. 5: Good prediction



Fig. 6: Inaccurate prediction

Limitations:

- Limited generalization beyond the specific training dataset.
- Keypoint localization accuracy was insufficient for precise corner detection.
- Requires a large, diverse annotated dataset for robust performance.
- Heavy computational resources needed for training and inference.
- Post-processing is necessary to refine noisy or missing keypoints.

E. Top-Down Contour-Based Chessboard Detection (Chosen approach)

This was the final chosen approach. This method is contour-based but follows a top-down strategy rather than building up from detected edges. It assumes the chessboard is fully visible within the image, roughly centered, and occupies a significant portion of the frame. The approach starts by identifying the largest quadrilateral contours in the image, which serve as candidate chessboards.

For each candidate, a projection is applied to straighten the quadrilateral region. Then, two morphological filters are applied separately to isolate vertical and horizontal lines, effectively removing non-grid elements such as chess pieces. With this cleaned-up candidate, the Hough transform is used to detect grid lines, and the pipeline continues similarly to the contour-based detection approach to extract the grid and corners.

Detailed Step-by-Step Process

- **Grayscale Conversion:** Convert the input image to grayscale.
- **Bilateral Filtering:** Apply a bilateral filter to reduce noise while preserving edge sharpness.
- **Canny Edge Detection:** Generate a binary edge map using the Canny edge detector.

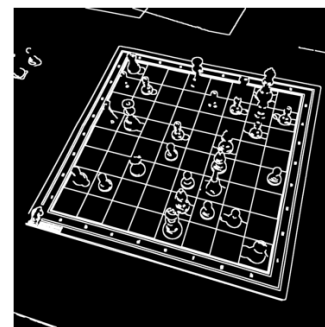


Fig. 7: Canny edge operator

- **Finding Large Contours:** Search for contours with areas approximately 50 to 70 times the expected area of a single chessboard cell. This heuristic typically returns two or three candidate contours, one of which contains the entire chessboard. Note that these candidate contours are intentionally larger than the actual chessboard to ensure full coverage and avoid partial detections.

- **Processing Each Candidate Contour:**

- **Projection:** Apply a geometric projection to warp the contents within the candidate's quadrilateral contour into a normalized, axis-aligned rectangle. This step aligns the chessboard so that most edges become vertical or horizontal.

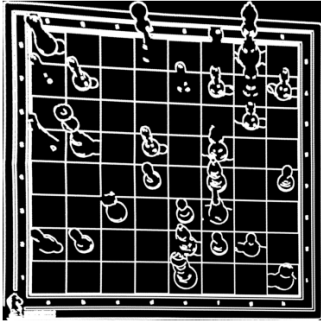


Fig. 8: Candidate canny projection

- **Vertical and Horizontal Masking:** Use separate morphological kernels to isolate vertical and horizontal lines. This filtering reduces noise caused primarily by the rounded shapes of chess pieces, enhancing grid line clarity.

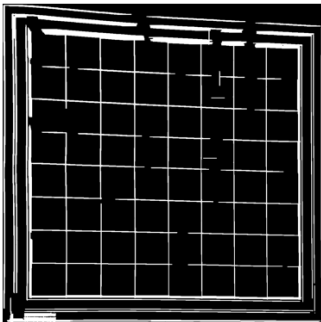


Fig. 9: Vertical and horizontal detected lines

- **Line Extension and Cleanup:** Perform a Probabilistic Hough Line Transform to detect and extend lines that are at least 80% the length of a single chessboard square and fill gaps up to 50% of the expected square length. This generates a clean grid representation of the projected chessboard.

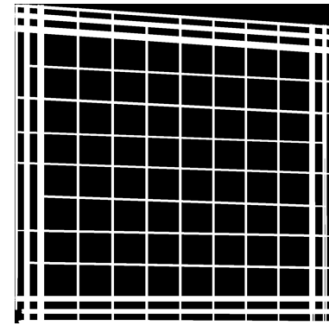


Fig. 10: Mask cleanup

- **Square Extraction:** Detect contours on the cleaned image and filter them strictly to identify squares that correspond to individual chessboard cells.

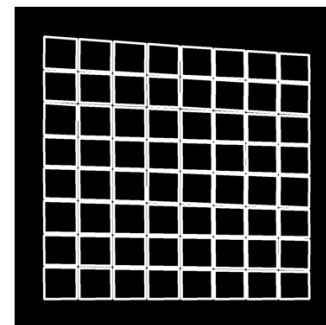


Fig. 11: Mask cleanup

- **Contour and Corner Extraction:** Extract the largest contour that encloses all detected squares and approximate it to a quadrilateral to estimate the chessboard boundary. Among multiple candidates, select the one with the highest number of detected squares and best centrality within the image.

- **Mapping the Chessboard Grid:** With the estimated corners, divide the quadrilateral into a regular 8x8 grid. This grid defines the bounding boxes of all chessboard cells, which are used in subsequent steps such as chess piece detection and recognition.



Fig. 12: Projection to original image

Limitations:

- Relies on strong assumptions: chessboard must be fully visible and centered.
- Fails if the chessboard is partially outside the image or very small.
- Sensitive to contour detection errors—largest quadrilateral might not be the board.
- Less effective in cluttered scenes with multiple large quadrilaterals.

F. Method comparison

To have an objective measure of performance between methods we defined two useful metrics:

- 1) **Average time:** average time that is needed to compute the corners given an image
- 2) **Accuracy:** a detection is accurate if all detected corners land no more than 30 pixels from the ground truth.

Having defined the metrics, by utilizing chessred2k dataset [1], containing 2078 labeled images we were able to compute a benchmark of the proposed methods. (run on intel i7-10700K CPU, RAM 32GB, RTX 2080 SUPER).

TABLE I: Performance comparison of the proposed methods on chessred2k dataset [1].

Method	Average time	Accuracy
Line detection and clustering	N/A	N/A
Contour-Based	~0.46s	24.06%
Parallel Multi-Parameter Contour-Based	~5.154s	90.91%
Deep Learning Corner Keypoint	~0.013s	45.04%
Top-Down Contour-Based Chessboard	~0.312s	92.16%

III. CHESS-PIECES OBJECT DETECTION

Our approach in the object detection has been a global one. We chose to use a global approach in order to not segment every image in single pieces. In this way we could avoid having the empty class and the duplication of object detected that could have been present thanks to the duplication of the piece due to overlaps created during the image extraction (cropping) process.

We initially approached the problem of chess-pieces object detection using DETR (Detection Transformer), with a ResNet-50 backbone, used for object detection tasks. The DETR model was pre-trained on the COCO dataset and fine-tuned on a custom dataset of 2000 chess images, which included various chess pieces in different lighting conditions, angles, and designs annotated in the COCO format with 12 classes representing the different chess pieces (e.g., white-pawn, white-knight, white-bishop, etc.). However, we encountered several challenges with this method, particularly in terms of performance and accuracy. Despite tuning and training the model for 100, 150 and 200 epoch, DETR consistently underperformed. The DETR model struggled with the variability of chess pieces in different lighting conditions, angles, and designs, leading to suboptimal detection results. We found out that the model was not able to converge properly, and the loss function did not decrease as expected during training. This

led us to explore alternative approaches for chess-piece object detection.

We then turned our attention to YOLOv8, a state-of-the-art object detection model that has shown promising results in various applications. We trained YOLOv8 on the same custom dataset of 2000 chess images, which included various chess pieces in different lighting conditions, angles, and designs. The training process was more successful with YOLOv8, as the model was able to converge properly and achieve a lower loss function. The YOLOv8 model demonstrated superior performance in terms of accuracy and robustness compared to DETR.

The model was trained from a pre-trained checkpoint found online and fine-tuned on our custom dataset.[ref to github repo] We conducted three training runs with different parameters like:

- Data augmentations (mosaic augmentation)
- HSV shifts
- Horizontal and vertical flips

The training was performed using PyTorch 2.1 on an NVIDIA RTX 3090 GPU with 24GB of VRAM.

It has been a difficult task to train the model in the right way. We tried to fine tune the model found in different ways using some datasets found online. The three training runs were conducted with the two datasets in this order:

- YOLOv8m run 1: A small dataset found on Roboflow, which contains 600+ images of chess pieces in different lighting conditions and angles. [10]
- YOLOv8m run 2: ChessRed2K dataset [1], which contains 2000 images of chess pieces in various conditions.
- YOLOv8m run 3: Starting from run 1 we used ChessRed2K dataset for finetuning the model.

As we can see in the table below, the model achieved high precision and recall values across the three training runs, indicating its effectiveness in detecting and classifying chess pieces. All the runs showed converged approximately in 200 epochs.

TABLE II: Key performance metrics for the three YOLOv8m training runs.

Run	Precision (B)	Recall (B)	mAP@0.5	mAP@0.5:0.95
YOLOv8m Run 1	~0.88	~0.93	~0.87	~0.75
YOLOv8m Run 2	~0.90	~0.95	~0.89	~0.78
YOLOv8m Run 3	~0.99	~0.99	~0.995	~0.99482

With the corresponding loss values:

TABLE III: Loss values for the three YOLOv8m runs. All values are approximate and taken at convergence (step ~200).

Run	Classification Loss (cls_loss)	Box Loss (box_loss)	DFL Loss (dff_loss)
YOLOv8m Run 1	~0.06	~0.35	~1.10
YOLOv8m Run 2	~0.05	~0.32	~1.05
YOLOv8m Run 3	~0.07	~0.38	~1.15

After the training runs, we used the best model from the third run of YOLOv8m for chess-piece object detection. We found out that the model has overfitted the training data, achieving very high precision and recall values on that dataset

but not performing well on unseen data.

After that we decided to find better datasets for training the model and started a training run from scratch. We found out multiple datasets online on Roboflow in order to start multiple training runs and find the best model for chess-piece object detection. The datasets used for training YOLOv8m are summarized in table below.

TABLE IV: Datasets used for training YOLOv8m.

Dataset Name	Nr Images	Nr Training Images	Nr Validation Images	Nr Test Images
Merge Chess [3]	~19708	~18141	~1512	~55
Chess Piece Detection [6]	~14808	~12194	~1743	~871
Piece Detector [2]	~18690	~16821	~1121	~748

After many training runs we found out that the best model was with the second dataset, which was trained for 200 epochs with a batch size of 16 and image size of 640 and an initial learning rate set by the model. We used the YOLOv8m model, which is a medium-sized version of the YOLOv8 architecture, suitable for our chess-piece detection task.

We tried to use that model but we could not achieve the desired results. Then after many researches on the web and on ultralytics documentation we found out that the model was not trained properly. We decided to use the YOLOv8m model with the following parameters:

- Batch size: 32
- Image size: 640
- Initial learning rate: 0.01
- Number of epochs: 1200
- Patience: 100

We started the first training with only one dataset, Chess Piece Detection [6], which contains 14808 images of chess pieces in various conditions. After 600 epochs we achieve some good results, but not good enough for our task. We decided to merge all the datasets found online and start a new training run with the merged dataset. The merged dataset contains 53206 images of chess pieces in various conditions, which is a significant increase in the amount of training data compared to the previous runs. We trained the model for 1200 epochs with a batch size of 32 and an image size of 640.

After 200 epochs we started to see some good results in the training process, with the loss function decreasing and the precision and recall values increasing even more than the previous runs.

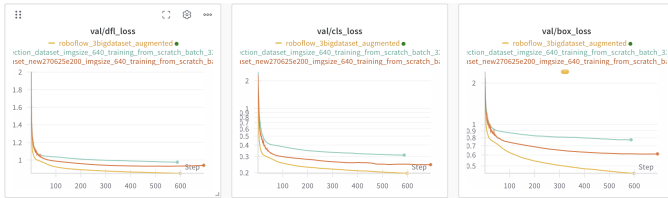


Fig. 13: Training loss and metrics for the merged dataset (validation set).

We found out that near 700 epochs the model had a good checkpoint, it had an overall accuracy of 94% when the chessboard was full of pieces. Instead of stopping the

training we tried to let it continue for more epochs in order to understand if we could achieve better results even if the current one was the best at that moment. After 200 more epochs we saw that the training and the validation loss were still going down but after some test with that checkpoints we found out that the model overfitted.

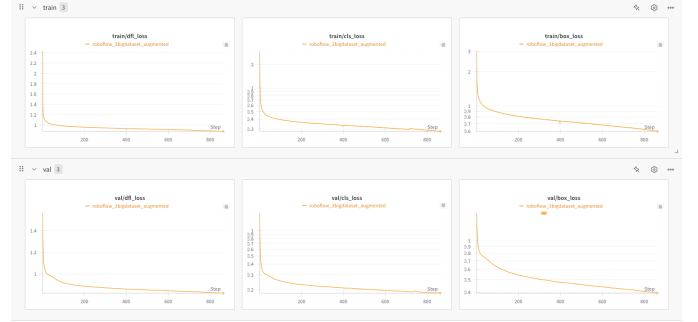


Fig. 14: Training and Validation loss and metrics for the merged dataset.

If we have to take into account some metrics for the good checkpoint we could see that our precision-recall was pretty good.

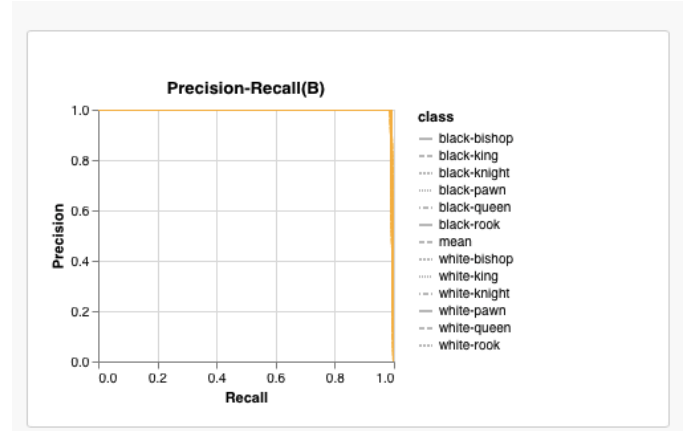


Fig. 15: Precision-Recall on merged dataset

In terms of loss behavior, we decided to use three different loss functions:

- Classification Loss: it measures the accuracy of the model predictions in classifying the class inside the bounding box. For this loss YOLOv8m uses a binary cross-entropy loss function.
- Box Loss: it measures the accuracy of the model in predicting bounding box positions and sizes. It's calculated with a combination of IoU (Intersection over Union).
- Distribution Focal Loss: this loss function is an advanced feature of YOLOv8. It measures the accuracy of the model in identifying the presence of objects inside the image. It's used to handle better small objects.

Compared to earlier training runs and attempts with DETR that showed high loss values and poor convergence,

TABLE V: Final validation loss values for the merged dataset. All values are approximate and taken at convergence

Run	Classification Loss (cls_loss)	Box Loss (box_loss)	DFL Loss (dfl_loss)
CPD Run 1 - 600epochs	~0.97525	~0.31069	~0.77366
Merged Dataset Run 1 - 600epochs	~0.84877	~0.19713	~0.44311
Merged Dataset Run 2 - 800+epochs	~0.8247	~0.18167	~0.39219

YOLOv8m performed significantly better in terms of loss values and detection robustness. We decided to use the third run of YOLOv8m as our final model for chess-piece object detection, as it showed the best performance in terms of precision, recall, and mAP values.

At the end of our research we used ChessRed2k as validation dataset for all the trainings done. We have saved up to 5 checkpoints (4 from our training and 1 found online) and estimated their capabilities. For the one found online, we talked to the owner and asked if we could use it and also if he could give us the same dataset but we couldn't reach the same level of accuracy.

TABLE VI: Final validation loss values for all the trainings done.

Run	P	R	mAP50	mAP50-95
First Run - Small Dataset	~0.372	~0.218	~0.202	~0.115
Only Chess Piece Detection Dataset	~0.827	~0.798	~0.873	~0.681
Three Merged Dataset - From Scratch	~0.874	~0.855	~0.913	~0.700
Three Merged Dataset - From PreTrained	~0.833	~0.847	~0.911	~0.735
CheckPoint Online	~0.921	~0.907	~0.964	~0.718

After getting our object detection done with the best checkpoint we could achieve we had to merge that information with our application. Easily speaking we have got 64 contours of the chessboard cells and the boxes given by the object detection. For every box we calculated a heuristic scaled point at 30px from the base of the box, after that we check if the calculated point could be found in one of the 64 contours. If the check is true we save the predicted class in the correct piece and color and put that piece in the correct position otherwise we check another point and so on. With this method we can reconstruct the digital chessboard state from the image given.

IV. FEN ANNOTATION METHOD

A. What is FEN?

"FEN is the abbreviation of Forsyth-Edwards Notation, and it is the standard notation to describe positions of a chess game. Steven J. Edwards, a computer programmer, created this notation system based on another system designed by the journalist David Forsyth." [5]

it allows easy storage and shareability of a position accounting for all the possible hidden states other than piece locations.

B. Definition

Composed by 6 ASCII strings separated by a space, each describes an aspect of the position.

- **Piece placement:** begins from the eighth rank and first file, describes the content of each square using lowercase letters for black, uppercase for white and numbers for empty squares.

Initial position

rnbqkbnr/pppppppp/8/8/8/PPPPPPPP/RNBQKBNR

- **Active color:** who is going to move next, denoted by "w" or "b"

rnbqkbnr/pppppppp/8/8/8/PPPPPPPP/RNBQKBNR w

- **Castling rights:** Uppercase letter comes first and indicates castling right for white, followed by lowercased for black, "k" indicates king side is available, "q" means that a player may castle queenside

rnbqkbnr/pppppppp/8/8/8/PPPPPPPP/RNBQKBNR w KQkq

- **Possible En Passant Targets:** if a pawn has moved two squares it is possible to capture en passant, if available the FEN adds the square behind the pawn in algebraic notation, else a "-"

rnbqkbnr/pppppppp/8/8/8/PPPPPPPP/RNBQKBNR w KQkq -

- **Halfmove Clock:** Informs of how many moves both players have made since the last pawn advance or piece capture.

rnbqkbnr/pppppppp/8/8/8/PPPPPPPP/RNBQKBNR w KQkq - 0

- **Fullmove Number:** Shows the number of completed turns in the game.

rnbqkbnr/pppppppp/8/8/8/PPPPPPPP/RNBQKBNR w KQkq - 0 1

The ease of understanding and simplicity of the format makes it trivial to implement a parser, thus allowing almost global use of the format across chess related software being the de factor standard to describe a chess position.

V. IMAGE SIMILARITY SEARCH

A. Current storage approaches

Although fen is a perfect medium for sharing, storing position and retrieving the exact match, due to its definition it makes comparison between similar but not equal positions using the format difficult without parsing and loading both positions in memory, becoming an hindrance especially when browsing chess databases to find games with similar position due to the computational cost of the operation.

The state of the art for storing and retrieving positions is varied based on the system that utilizes it. Most famously:

- For database used for human study, SCID Shane's Chess Information Database [12] one of the most famous tools used to find, navigate and study games, powerful but does not handle search by similarity.
- For storing high volume of data Lichess, one of the main chess platforms, has a developed a highly efficient format [11] that allows search alike SCID.

Even if highly efficient and perfect for their respective use cases, this formats do not suit the objective set for this project being the digitalization of physical chess position and search of similar position.

The inability to allow search by similarity is mainly because

of the subjective definition of similarity in games, as it could be approached from different points of view.

- **Pure Positional similarity:** position of pieces on the board
- **Threat similarity:** attacking and defending pieces
- **Structural similarity:** in most games recurring piece structures are created

A novel approach that tries to merge most of the aspects related to similarity was explored by Ganguly, Debasis and Leveling, Johannes and Jones, Gareth 2014 [7] that offers a novel approach by using information retrieval methods applied to a position. By modeling a text representation of the position utilizing positional information, square control, attacking and defending pieces to be used inside the standard inverted organization typical of IR.

In this project we utilized only positional information to build a pure piece positional index similarity, sacrificing similarity accuracy for improved performance.

B. Our method

We opted for a pure positional information approach, although using only the position of the pieces may return only apparently similar games with them being different, because of slight changes in position could lead to big difference in play, the approach is valid to explore the problem and given a sufficiently large database it will return games in which the position or a position less than two move difference.

Given any position, it is encoded in a naive binary embedding of 64 slot of length 12 bits where the first 6 bits represent the white pieces and the other 6 bits the black pieces.



Fig. 16: Knight to embedding

This results in a sparse binary vector embedding of length 96 bytes that is an exact one to one of the whole piece positional information of the given position.

Algorithm 1 Embedding algorithm

```

v ← (96B array)
P ← Board position
for each s ∈ P do
  if s is not empty then
    i ← computeSquareIndex(s)
    v[i] ← 1
  end if
end for

```

C. Search database creation

As our chess database of choice we chose LumbrasGigabase [8], containing 15 Million games from various sources counting to approximately 1 billion unique game positions

reached, stored in a standard format called PGN (portable game notation) [4] supplying game moves and metadata relative to each game useful to populate our database.

```

[Event "F/S Return Match"]
[Site "Belgrade, Serbia JUG"]
[Date "1992.11.04"]"newline[Round "29"]
[White "Fischer, Robert J."]
[Black "Spassky, Boris V."]
[Result "1/2-1/2"]

```

```

1.e4 e5 2.Nf3 Nc6 3.Bb5 This opening is called
the Ruy Lopez. 3...a6 4.Ba4 Nf6 5.O-O Be7 6.Re1
b5 7.Bb3 d6 8.c3 O-O 9.h3 Nb8 10.d4 Nbd7 11.c4
c6 12.cxb5 axb5 13.Nc3 Bb7 14.Bg5 b4 15.Nb1 h6
16.Bh4 c5 17.dxe5 Nxe4 18.Bxe7 Qxe7 19.exd6 Qf6
20.Nbd2 Nxd6 21.Nc4 Nxc4 22.Bxc4 Nb6 23.Ne5 Rae8
24.Bxf7+ Rxf7 25.Nxf7 Rxe1+ 26.Qxe1 Kxf7 27.Qe3
Qg5 28.Qxg5 hxg5 29.b3 Ke6 30.a3 Kd6 31.axb4 cxb4
32.Ra5 Nd5 33.f3 Bc8 34.Kf2 Bf5 35.Ra7 g6 36.Ra6+
Kc5 37.Ke1 Nf4 38.g3 Nxh3 39.Kd2 Kb5 40.Rd6 Kc5
41.Ra6 Nf2 42.g4 Bd3 43.Re6 1/2-1/2

```

Bobby Fischer vs Boris Spassky, 1992, Yugoslavia, 29th

By iterating over each game we extract and process each position by creating its embedding and Zobrist hash [13], de factor standard hash used to encode a chess position in 64 bit, to efficiently store the same position used in different games. Although we had access to a large quantity of high quality games due to hardware memory and computation limitations we had to scale back the indexing of the positions to allow only 5 million instances.

D. Distance function

The search and retrieval of the embeddings is efficiently done through an hamming distance HNSW index [9], a fully graph based index without any need for additional search structures, extension of earlier work on navigable small world graphs.

The cited index enables an efficient ranking of the saved positions based on the number of flipped bits compared to the anchor embedding.

While not applicable universally we observed that a unit distance of 2 corresponds to a single different piece position, this could be intuitively understood by seeing it as toggling squares, one off square and one on square.

This does not generalize in case of captures, castling, en passant, or any move that involve more than one piece.

Through the use of the HNSW index we are able to have a considerable speedup in search and retrieval speed on 10 million indexed positions. By creating the binary embedding, chess database and vector index we have efficiently implemented a retrieval system for similar games able to return the closest game in terms of number of moves and piece on the board to a given position.

TABLE VII: Speed comparison with and without vector index.

Run	Average time
HNSW	~121ms
None	~959ms

VI. CONCLUSION

In this paper, we tried to bring a comprehensive framework for detecting 3D chessboards and identifying individual chess pieces in real-world chess games. The approach proposed combines traditional computer vision techniques with deep learning methods such as YOLOv8m to address the multiple challenges of variable lighting, perspective distortion and heterogeneous piece design recognition.

At the end of our study, we selected a top-down contour-based strategy that demonstrated superior robustness under real-world constraints. About the object detection part, we conducted extensive experimentation with both transformed-based and convolutional architectures, determining that YOLOv8m, trained on a large dataset, offers the most accurate and reliable performance for the task of chess-piece classification.

After all the recognition part, we developed a practical pipeline for generating FEN (Forsyth-Edwards Notation) notation from a board state, enabling seamless conversion from a physical board states to a standard digital representation. In addition, we proposed a binary embedding-based similarity search method for retrieving similar state games from a large dataset of millions of games.

Despite the promising results, some limitation remains. Particularly the reliance on a full board visibility.

REFERENCES

- [1] 4TU.ResearchData. Chess recognition dataset (chessred2k), 2023. Online; accessed 2025-06-29.
- [2] cgeess. Piece detector computer vision project, 2024. Online; accessed 2025-06-29.
- [3] ChDuck. Merge chess computer vision project, 2024. Online; accessed 2025-06-29.
- [4] Chess.com. Portable game notation (pgn), 2023. Online; accessed 2025-06-28.
- [5] Chess.com. Forsyth-edwards notation (fen), 2024. Online; accessed 2025-06-28.
- [6] Chess Piece Detection. Chess piece detection computer vision project, 2025. Online; accessed 2025-06-29.
- [7] Debasis Ganguly, Johannes Leveling, and Gareth Jones. Retrieval of similar chess positions. 07 2014.
- [8] Lumbras. Lumbrasgigabase, 2025. Online; accessed 2025-05-12.
- [9] Yu A. Malkov and D. A. Yashunin. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 42(4):824–836, 2020.
- [10] ML Reasearch. Chess yolo computer vision project, 2022. Online; accessed 2025-06-29.
- [11] Revoof. Adapting nnue-pytorch’s binary position format for lichess, 2024. Online; accessed 2025-06-28.
- [12] SCID. Scid. Online; accessed 2025-06-28.
- [13] Albert Zobrist. A new hashing method with application for game playing. technical report #88. 1970. ICCA Journal, Vol. 13, No. 2.